LabPulse AI: Intelligent Laboratory Resource Monitoring and Anomaly Detection Using Isolation Forest

Problem Statement:

Laboratories use computer systems and digital resources to perform academic, research, and computational activities. High CPU usage, excessive memory consumption, unusual network activity, and a large number of running processes may affect system performance. Manually monitoring these resources is time-consuming and may not identify unusual resource usage quickly.

There is, therefore, a need for an intelligent system that can monitor laboratory resource usage, identify unusual patterns, and provide useful recommendations. This project develops LabPulse AI, a web-based laboratory resource monitoring and anomaly detection system using the Isolation Forest algorithm. The system analyses resource usage data, identifies normal and anomalous records, calculates a dashboard risk indicator, and displays the results through an interactive web interface.

Objectives:

1. To generate and analyse laboratory resource usage data.
2. To preprocess the dataset and select relevant resource usage features.
3. To develop an anomaly detection model using the Isolation Forest algorithm.
4. To identify normal and anomalous laboratory resource usage records.
5. To develop an interactive dashboard using Streamlit and Plotly.
6. To analyse laboratory resource usage and display anomaly detection results with recommendations.

Software / Hardware Required:

• Operating System: Windows
• IDE: Visual Studio Code
• Programming Language: Python 3.x
• Machine Learning Library: scikit-learn
• Data Processing: Pandas, NumPy
• Data Visualization: Matplotlib, Seaborn, Plotly
• Web Framework: Streamlit
• Model Storage: Joblib
• Dataset: Synthetic laboratory resource usage dataset

• Hardware: Computer with a suitable processor and sufficient RAM

Project Directory Structure:

The project ("LabPulse AI") is organized into Python modules with separate data, model, utility, and asset folders, as shown below.

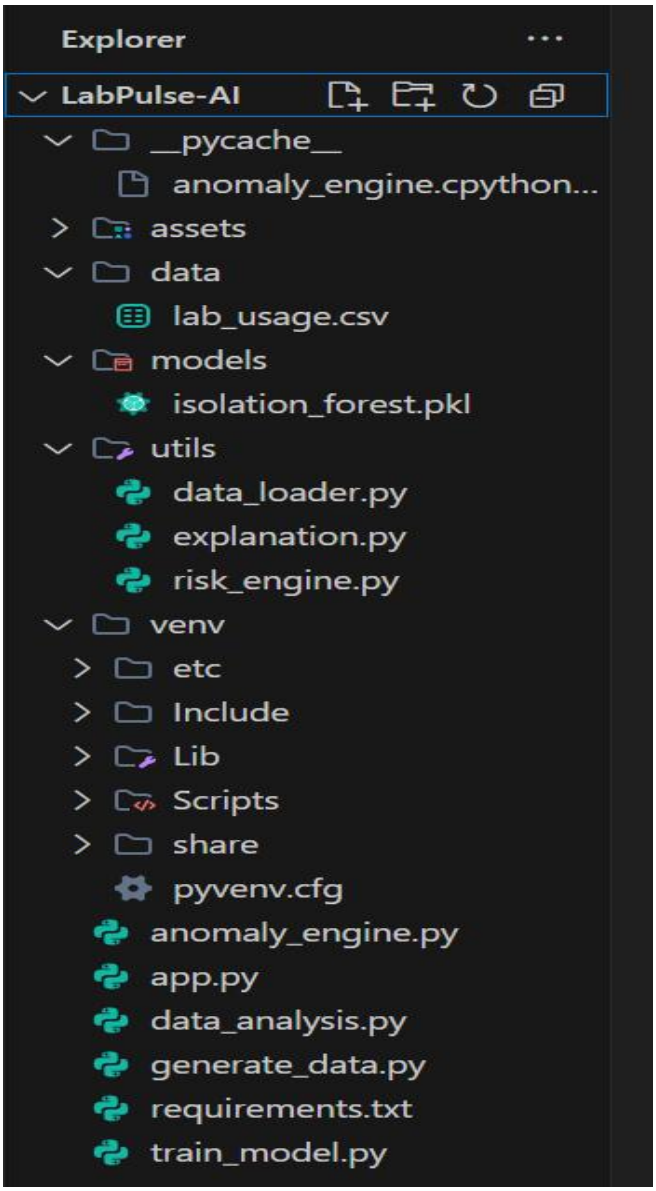


Fig 1: Project directory structure of LabPulse AI in Visual Studio Code

Procedure:

1. Generate a laboratory resource usage dataset containing 5,000 records across four laboratories using generate_data.py.

2. Load and analyse the dataset using data_analysis.py to examine resource usage, missing values, duplicate records and statistical summaries.

3. Preprocess the dataset by selecting relevant features, including CPU Usage, RAM Usage, Disk Usage, Network Usage, Running Processes, Active Users and Session Duration.

4. Standardize the selected features using StandardScaler to prepare the data for machine learning.

5. Develop an unsupervised anomaly detection model using the Isolation Forest algorithm with 200 estimators and an 8% contamination parameter.

6. Train the model using train_model.py and save the trained model as models/isolation_forest.pkl.

7. Identify normal and anomalous laboratory resource usage records using the trained model, and save the detection results in data/lab_usage.csv.

8. Calculate the resource stress score, determine the risk level and generate recommendations based on the laboratory resource usage conditions using anomaly_engine.py.

9. Develop an interactive dashboard using Streamlit and Plotly to display resource utilization, AI analysis results, anomaly history and laboratory performance statistics.

10. Evaluate and visualize the results by displaying normal records, detected anomalies, anomaly rates, resource usage trends and laboratory comparisons through the LabPulse AI dashboard.

Implementation Code:

Representative implementation of the key modules used in the LabPulse AI project for laboratory resource usage analysis and anomaly detection.

1. Dataset Generation (generate_data.py)

```
import pandas as pd
import numpy as np
np.random.seed(42)
labs = ["LAB-01", "LAB-02", "LAB-03", "LAB-04"]
n = 5000
df = pd.DataFrame({
    "Timestamp": pd.date_range(
        "2026-01-01", periods=n, freq="h" ),
    "Lab_ID": np.random.choice(labs, n),
    "CPU_Usage": np.random.uniform(20, 100, n),
```

```
        "RAM_Usage": np.random.uniform(25, 100, n),
        "Disk_Usage": np.random.uniform(20, 95, n),
        "Network_Usage": np.random.uniform(10, 100, n),
        "Running_Processes": np.random.randint(10, 100, n),
        "Active_Users": np.random.randint(1, 40, n),
    "Session_Duration": np.random.randint(10, 300, n)
    })
df.to_csv("data/lab_usage.csv", index=False)
print("Dataset created successfully!")
print("Total Records:", len(df))
print("Total Labs:", df["Lab_ID"].nunique())
```

2. Anomaly Detection Model (train_model.py)

```
import pandas as pd
from sklearn.ensemble import IsolationForest
df = pd.read_csv("data/lab_usage.csv")
features = [
        "CPU_Usage", "RAM_Usage", "Disk_Usage",
        "Network_Usage", "Running_Processes",
        "Active_Users", "Session_Duration"]
model = IsolationForest(
        n_estimators=200,
        contamination=0.08,
        random_state=42)
df["AI_Result"] = model.fit_predict(df[features])

df["AI_Status"] = df["AI_Result"].map({
        1: "Normal",
        -1: "Anomaly"
})
print(df["AI_Status"].value_counts())
```

3. Interactive Dashboard (app.py)

```
import streamlit as st
import pandas as pd
import plotly.express as px
st.title("LabPulse AI")
df = pd.read_csv("data/lab_usage.csv")
```

```
st.metric("Total Records", len(df))
st.metric("Anomalies",
          (df["AI_Status"] == "Anomaly").sum())
fig = px.scatter(
    df,
    x="CPU_Usage",
    y="RAM_Usage",
    color="AI_Status",
    title="CPU vs RAM Usage"
)
st.plotly_chart(fig, use_container_width=True)
```

4. Risk Score Calculation (anomaly_engine.py)

```
def calculate_risk(cpu, ram, disk, network):
    stress = (cpu + ram + disk + network) / 4

    if stress >= 75:
        level = "CRITICAL"
    elif stress >= 50:
        level = "HIGH"
    elif stress >= 25:
        level = "MEDIUM"
    else:
        level = "LOW"

    return stress, level
```

5. Saving Anomaly Detection Results

```
import pandas as pd

def save_results(df):
    df.to_csv(
        "data/lab_usage.csv",
        index=False
    )
    print("Anomaly results saved successfully!")
```

Output (Step-by-Step Screenshots):

```
========================================
       LabPulse AI - Dataset Generator
========================================

Dataset created successfully!
Records       : 5000
Labs          : 4
Features      : 9
File          : data/lab_usage.csv

First 5 Records:
                 Timestamp  Lab_ID  CPU_Usage  ...  Running_Processes  Active_Users  Session_Duration
0 2026-09-11 14:08:40.178602  LAB-03      85.46  ...                 52             4               188
1 2026-09-11 14:18:40.178602  LAB-04      86.66  ...                 48            30               163
2 2026-09-11 14:28:40.178602  LAB-01      45.94  ...                 37            35                32
3 2026-09-11 14:38:40.178602  LAB-03      95.74  ...                 18             9               138
4 2026-09-11 14:48:40.178602  LAB-03      56.60  ...                 57             8               132

[5 rows x 9 columns]

Dataset Statistics:
                           Timestamp    CPU_Usage  ...  Active_Users  Session_Duration
count                           5000  5000.000000  ...   5000.000000       5000.000000
mean   2026-09-28 22:43:40.178601984    61.084724  ...     20.115800        135.779400
min       2026-09-11 14:08:40.178602     5.000000  ...      1.000000         10.000000
25%    2026-09-20 06:26:10.178601984    46.355000  ...      9.000000         78.000000
50%    2026-09-28 22:43:40.178601984    61.015000  ...     20.000000        138.000000
75%    2026-10-07 15:01:10.178601984    76.627500  ...     30.000000        193.000000
max       2026-10-16 07:18:40.178602   100.000000  ...     40.000000        299.000000
std                               NaN    20.233987  ...     11.948345         68.804523

[8 rows x 8 columns]

========================================
```

Fig 2: Dataset Generation: generating 5,000 laboratory resource usage records across four laboratories.

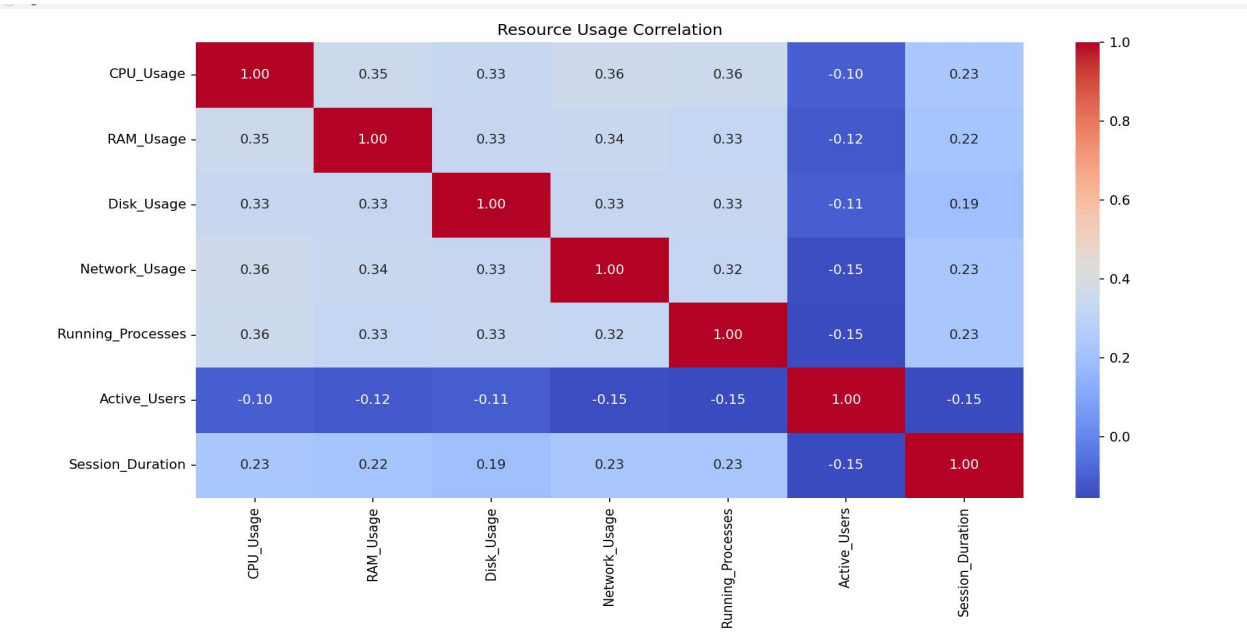


Fig 3:Resource Usage Correlation - illustrating the correlation between CPU, RAM, disk, network, active users, running processes, and session duration.

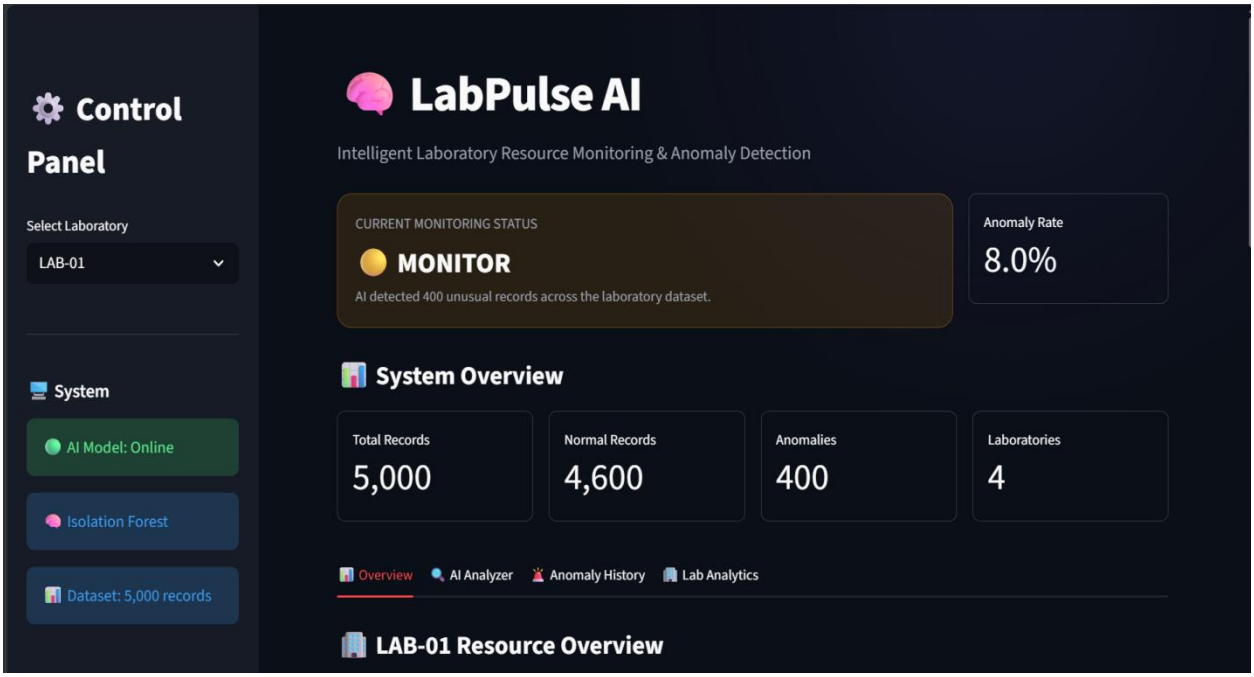


Fig 4: Main Dashboard Overview - displaying the monitoring status, total records, normal records, detected anomalies, and number of laboratories.

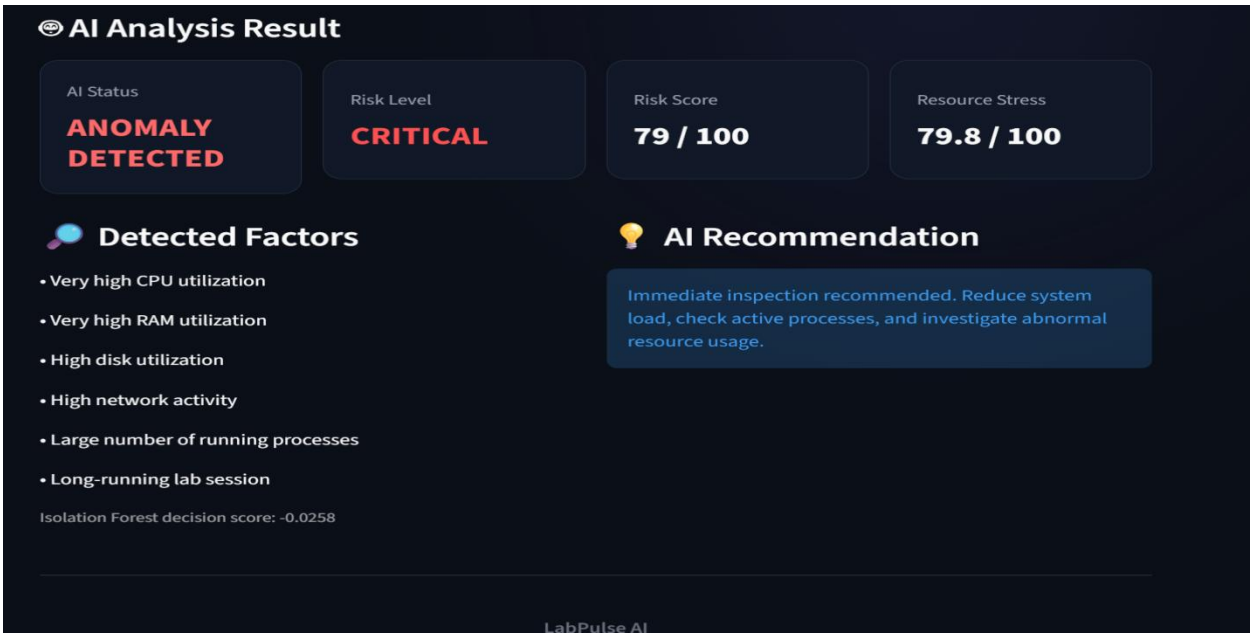


Fig 5: AI Analysis Result - displaying anomaly status, critical risk level, risk score, resource stress, detected factors, and AI recommendations.

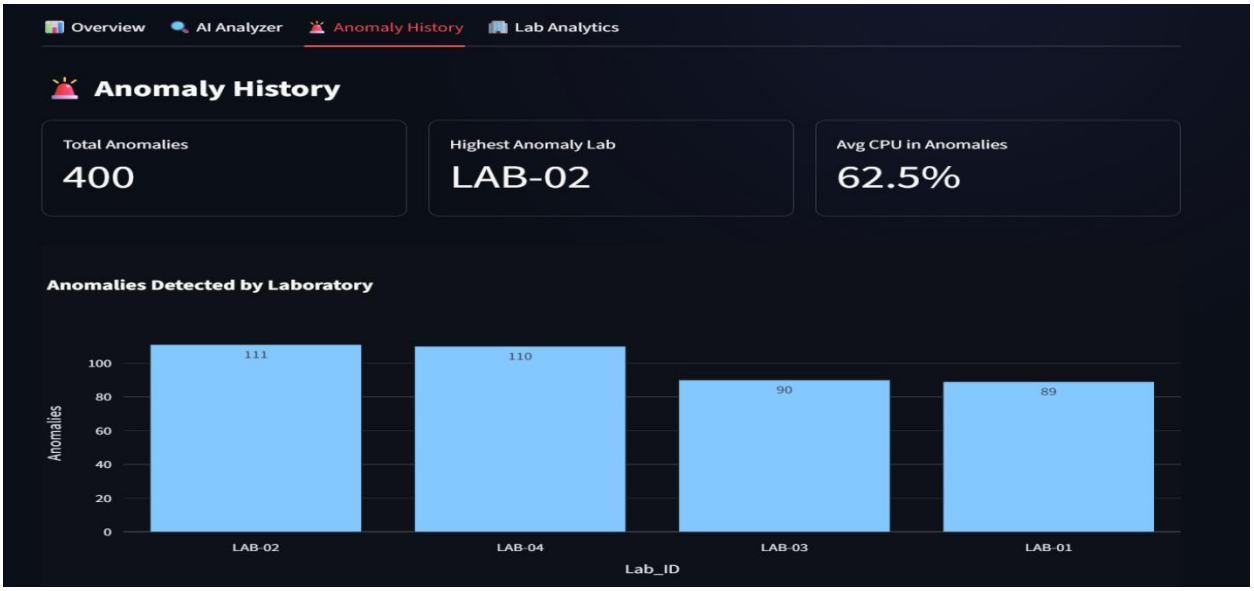


Fig 6: Anomaly History - displaying the total number of anomalies, the laboratory with the highest anomaly count, average CPU usage in anomalies, and laboratory-wise anomaly distribution.

Detected Anomaly Records

Timestamp	Lab_ID	CPU_Usage	RAM_Usage	Disk_Usage	Network_Usage	Running_Processes	Active_Users	Session_Dur
2026-10-04 14:20:26.802221	LAB-01	69.94	19.12	74.57	70.09	60	35	
2026-10-04 14:30:26.802221	LAB-03	56.26	15.15	5	62.65	17	3	
2026-10-04 15:10:26.802221	LAB-02	73.39	90.13	31.19	72.7	26	7	
2026-10-04 15:20:26.802221	LAB-01	96.82	94.24	97.4	81.91	87	5	
2026-10-04 16:50:26.802221	LAB-04	62.87	34.68	48.27	91.52	11	32	
2026-10-04 17:20:26.802221	LAB-02	94.39	92.61	86.2	93.89	81	4	
2026-10-04 21:00:26.802221	LAB-01	54.27	78.32	19.23	100	21	5	
2026-10-04 21:10:26.802221	LAB-02	46.15	50.13	66.4	1	26	39	
2026-10-04 21:40:26.802221	LAB-01	100	80.64	42.27	45.45	30	37	
2026-10-04 22:00:26.802221	LAB-04	78.23	49.73	92.69	88.63	17	31	
2026-10-04 22:50:26.802221	LAB-03	60.54	100	28.95	31.44	23	33	

Fig 7: Detected Anomaly Records - displaying laboratory resource usage data identified as anomalous by the Isolation Forest model.

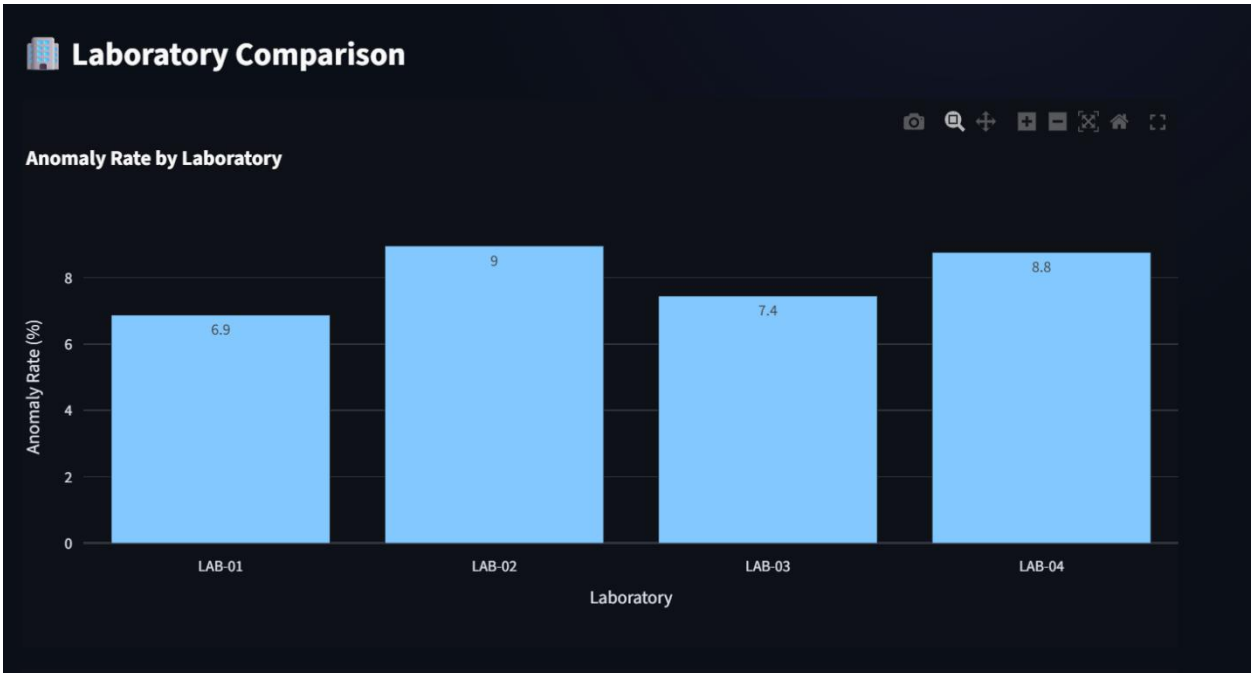


Fig 8: Anomaly Rate by Laboratory - comparing the percentage of detected anomalies across LAB-01, LAB-02, LAB-03, and LAB-04.

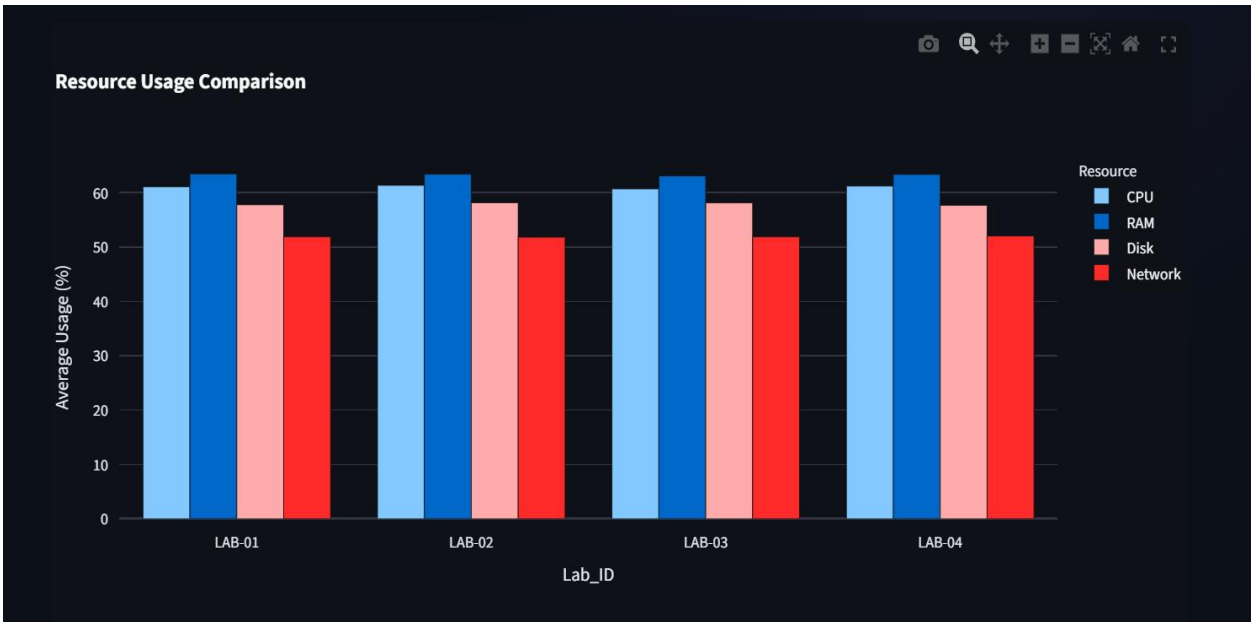
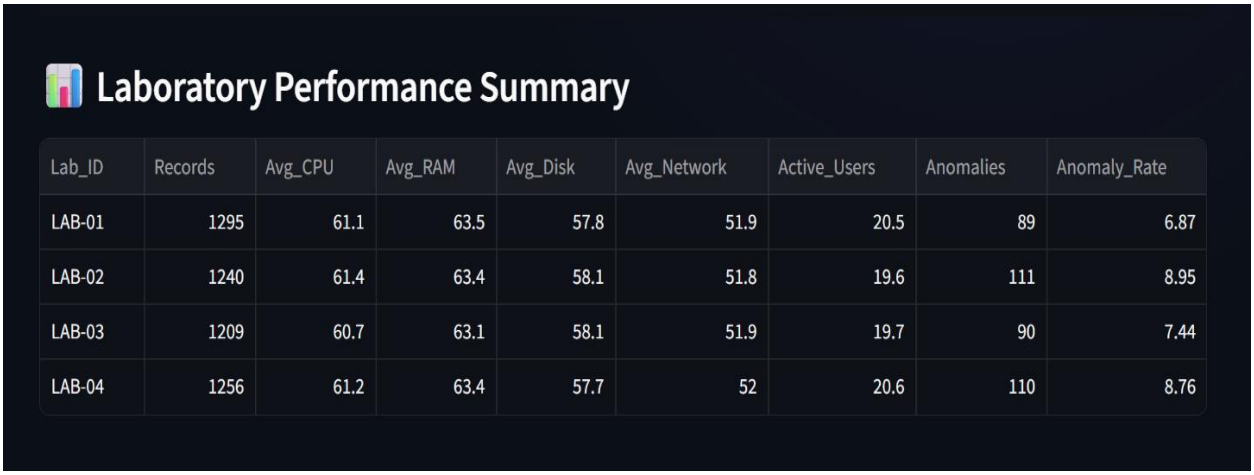


Fig 9: Resource Usage Comparison - comparing average CPU, RAM, disk, and network usage across four laboratories.



Laboratory Performance Summary

Lab_ID	Records	Avg_CPU	Avg_RAM	Avg_Disk	Avg_Network	Active_Users	Anomalies	Anomaly_Rate
LAB-01	1295	61.1	63.5	57.8	51.9	20.5	89	6.87
LAB-02	1240	61.4	63.4	58.1	51.8	19.6	111	8.95
LAB-03	1209	60.7	63.1	58.1	51.9	19.7	90	7.44
LAB-04	1256	61.2	63.4	57.7	52	20.6	110	8.76

Fig 10: Laboratory Performance Summary - displaying resource usage statistics, total records, anomalies, and anomaly rates for each laboratory.

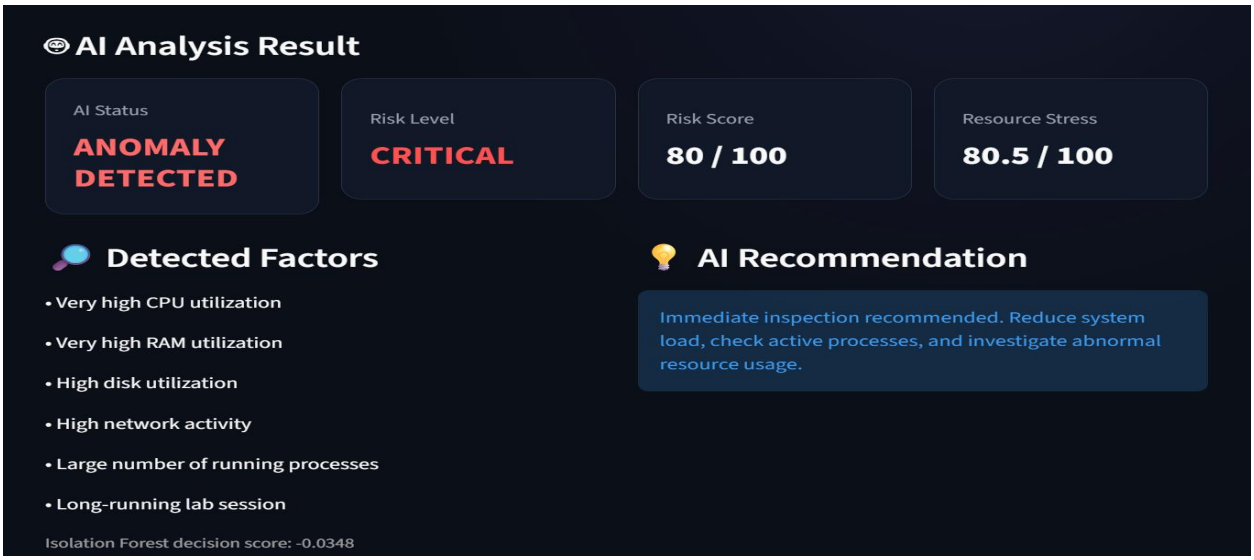


Fig 11: Critical Anomaly Detection - displaying the critical risk level, resource stress score, and recommended actions for abnormal laboratory resource usage.

Result:

The LabPulse AI system was successfully developed using Python, Isolation Forest, Streamlit, and Plotly to detect abnormal laboratory resource usage. It analysed 5,000 records across four laboratories, identifying 4,600 normal records and 400 anomalies, resulting in an 8% anomaly rate. The system also provides risk scores, resource stress levels, recommendations, and interactive visualizations to support laboratory resource monitoring. The dashboard enables users to compare laboratory performance and identify unusual resource usage patterns efficiently.